Data Structures

BS (CS) _Fall_2025

Lab_08 Task

Learning Objectives:

1. Stacks and Queues using Linked Lists in C++

Lab Task 1:

Scenario
You are developing a system for a hospital emergency room. Patients arrive with different urgency levels and must be treated based on the severity of their condition, not their arrival time. Your task is to implement a queue to manage patient flow efficiently.

Task Requirements
Create a priority queue where each patient is an object containing:

- Patient Name (string)
- Age (integer)
- Urgency Level (string: "Critical", "Urgent", "Standard")
- Counter (To record arrival order)

Core Functionalities to Implement

1. **Patient Management:**
    - enqueue(name, age, urgencyLevel) - Adds a new patient to the queue at the appropriate position based on their urgency level.
    - dnqueue () - Removes and returns the patient from the queue.
2. **Queue Inspection:**
    - displayQueue() - Displays all waiting patients by showing all their details.
    - isQueueEmpty() - Checks if the emergency room queue is empty.

Advanced Operations

- **Prioritization:** Ensure that within the same urgency level, patients are served based on their arrival time (earlier arrival first).
- getQueueSize() - Returns the total number of patients waiting.

Implementation Guidelines

- Implement a Patient class and a ERQueue class.
- The priority can be managed by assigning integer weights to urgency levels (e.g., Critical=3, Urgent=2, Standard=1).

Google Test Cases:

```
// Test enqueue single patient
TEST(ERQueueTest, EnqueueSinglePatient) {
    ERQueue<int> queue;
```

```
    queue.enqueue("John Doe", 45, "Critical");
    EXPECT_FALSE(queue.isQueueEmpty());
    EXPECT_EQ(queue.getQueueSize(), 1);
}

// Test dequeue single patient
TEST(ERQueueTest, DequeueSinglePatient) {
    ERQueue<int> queue;
    queue.enqueue("Jane Smith", 30, "Urgent");
    Patient<int> p = queue.dequeue();
    EXPECT_EQ(p.patientName, "Jane Smith");
    EXPECT_EQ(p.age, 30);
    EXPECT_EQ(p.urgencyLevel, "Urgent");
    EXPECT_TRUE(queue.isQueueEmpty());
}

// Test dequeue from empty queue
TEST(ERQueueTest, DequeueFromEmptyQueue) {
    ERQueue<int> queue;
    EXPECT_TRUE(queue.isQueueEmpty());
    Patient<int> p = queue.dequeue();
    EXPECT_EQ(p.patientName, "");
}

TEST(ERQueueTest, CriticalBeforeUrgent) {
    ERQueue<int> queue;
    queue.enqueue("Bob", 50, "Urgent");
    queue.enqueue("Alice", 35, "Critical");

    Patient<int> p1 = queue.dequeue();
    EXPECT_EQ(p1.patientName, "Alice");
    EXPECT_EQ(p1.urgencyLevel, "Critical");

    Patient<int> p2 = queue.dequeue();
    EXPECT_EQ(p2.patientName, "Bob");
    EXPECT_EQ(p2.urgencyLevel, "Urgent");
}

TEST(ERQueueTest, CriticalBeforeStandard) {
    ERQueue<int> queue;
    queue.enqueue("Charlie", 25, "Standard");
    queue.enqueue("Diana", 60, "Critical");

    Patient<int> p1 = queue.dequeue();
    EXPECT_EQ(p1.patientName, "Diana");
```

```
    EXPECT_EQ(p1.urgencyLevel, "Critical");

    Patient<int> p2 = queue.dequeue();
    EXPECT_EQ(p2.patientName, "Charlie");
    EXPECT_EQ(p2.urgencyLevel, "Standard");
}

TEST(ERQueueTest, UrgentBeforeStandard) {
    ERQueue<int> queue;
    queue.enqueue("Eve", 40, "Standard");
    queue.enqueue("Frank", 55, "Urgent");

    Patient<int> p1 = queue.dequeue();
    EXPECT_EQ(p1.patientName, "Frank");
    EXPECT_EQ(p1.urgencyLevel, "Urgent");

    Patient<int> p2 = queue.dequeue();
    EXPECT_EQ(p2.patientName, "Eve");
    EXPECT_EQ(p2.urgencyLevel, "Standard");
}

TEST(ERQueueTest, AllThreePriorities) {
    ERQueue<int> queue;
    queue.enqueue("Standard1", 30, "Standard");
    queue.enqueue("Urgent1", 45, "Urgent");
    queue.enqueue("Critical1", 60, "Critical");
    queue.enqueue("Standard2", 25, "Standard");
    queue.enqueue("Urgent2", 50, "Urgent");

    EXPECT_EQ(queue.dequeue().urgencyLevel, "Critical");
    EXPECT_EQ(queue.dequeue().urgencyLevel, "Urgent");
    EXPECT_EQ(queue.dequeue().urgencyLevel, "Urgent");
    EXPECT_EQ(queue.dequeue().urgencyLevel, "Standard");
    EXPECT_EQ(queue.dequeue().urgencyLevel, "Standard");
}

TEST(ERQueueTest, FIFOForCriticalPatients) {
    ERQueue<int> queue;
    queue.enqueue("Critical1", 40, "Critical");
    queue.enqueue("Critical2", 50, "Critical");
    queue.enqueue("Critical3", 35, "Critical");

    EXPECT_EQ(queue.dequeue().patientName, "Critical1");
    EXPECT_EQ(queue.dequeue().patientName, "Critical2");
    EXPECT_EQ(queue.dequeue().patientName, "Critical3");
```

```
}

TEST(ERQueueTest, FIFOForUrgentPatients) {
    ERQueue<int> queue;
    queue.enqueue("Urgent1", 30, "Urgent");
    queue.enqueue("Urgent2", 45, "Urgent");
    queue.enqueue("Urgent3", 28, "Urgent");
    queue.enqueue("Urgent4", 52, "Urgent");

    EXPECT_EQ(queue.dequeue().patientName, "Urgent1");
    EXPECT_EQ(queue.dequeue().patientName, "Urgent2");
    EXPECT_EQ(queue.dequeue().patientName, "Urgent3");
    EXPECT_EQ(queue.dequeue().patientName, "Urgent4");
}

TEST(ERQueueTest, FIFOForStandardPatients) {
    ERQueue<int> queue;
    queue.enqueue("Standard1", 20, "Standard");
    queue.enqueue("Standard2", 65, "Standard");
    queue.enqueue("Standard3", 42, "Standard");

    EXPECT_EQ(queue.dequeue().patientName, "Standard1");
    EXPECT_EQ(queue.dequeue().patientName, "Standard2");
    EXPECT_EQ(queue.dequeue().patientName, "Standard3");
}

TEST(ERQueueTest, FIFOWithMixedPriorities) {
    ERQueue<int> queue;
    queue.enqueue("Urgent1", 30, "Urgent");
    queue.enqueue("Critical1", 50, "Critical");
    queue.enqueue("Urgent2", 40, "Urgent");
    queue.enqueue("Critical2", 55, "Critical");
    queue.enqueue("Standard1", 25, "Standard");

    EXPECT_EQ(queue.dequeue().patientName, "Critical1");
    EXPECT_EQ(queue.dequeue().patientName, "Critical2");
    EXPECT_EQ(queue.dequeue().patientName, "Urgent1");
    EXPECT_EQ(queue.dequeue().patientName, "Urgent2");
    EXPECT_EQ(queue.dequeue().patientName, "Standard1");
}

TEST(ERQueueTest, QueueSizeIncrementsCorrectly) {
    ERQueue<int> queue;
    EXPECT_EQ(queue.getQueueSize(), 0);
```

```
    queue.enqueue("Patient1", 30, "Critical");
    EXPECT_EQ(queue.getQueueSize(), 1);

    queue.enqueue("Patient2", 40, "Urgent");
    EXPECT_EQ(queue.getQueueSize(), 2);

    queue.enqueue("Patient3", 50, "Standard");
    EXPECT_EQ(queue.getQueueSize(), 3);
}

TEST(ERQueueTest, QueueSizeDecrementsCorrectly) {
    ERQueue<int> queue;
    queue.enqueue("Patient1", 30, "Critical");
    queue.enqueue("Patient2", 40, "Urgent");
    queue.enqueue("Patient3", 50, "Standard");

    EXPECT_EQ(queue.getQueueSize(), 3);
    queue.dequeue();
    EXPECT_EQ(queue.getQueueSize(), 2);
    queue.dequeue();
    EXPECT_EQ(queue.getQueueSize(), 1);
    queue.dequeue();
    EXPECT_EQ(queue.getQueueSize(), 0);
}

TEST(ERQueueTest, MultipleEnqueueDequeueOperations) {
    ERQueue<int> queue;

    queue.enqueue("Patient1", 30, "Urgent");
    queue.enqueue("Patient2", 40, "Critical");
    EXPECT_EQ(queue.getQueueSize(), 2);

    queue.dequeue();
    EXPECT_EQ(queue.getQueueSize(), 1);

    queue.enqueue("Patient3", 50, "Standard");
    queue.enqueue("Patient4", 35, "Critical");
    EXPECT_EQ(queue.getQueueSize(), 3);

    EXPECT_EQ(queue.dequeue().patientName, "Patient4");
    EXPECT_EQ(queue.dequeue().patientName, "Patient1");
    EXPECT_EQ(queue.dequeue().patientName, "Patient3");

    EXPECT_TRUE(queue.isQueueEmpty());
}
```

```
TEST(ERQueueTest, InterleavedOperations) {
    ERQueue<int> queue;

    queue.enqueue("A", 30, "Standard");
    queue.enqueue("B", 40, "Critical");
    EXPECT_EQ(queue.dequeue().patientName, "B");

    queue.enqueue("C", 50, "Urgent");
    queue.enqueue("D", 35, "Critical");
    EXPECT_EQ(queue.dequeue().patientName, "D");

    queue.enqueue("E", 45, "Standard");
    EXPECT_EQ(queue.getQueueSize(), 3);
}
```

Lab Task 2

Scenario

Programming languages use a call stack to manage function calls and returns. When a function is called, a stack frame is pushed. When a function returns, its frame is popped. Your task is to simulate this behavior.

Task Requirements

Create a stack where each stack frame is an object containing:

- Function Name (string)
- Line Number of Call (integer, to simulate execution order)
- Number of Parameters(integer)

Core Functionalities to Implement

1. **Execution Simulation:**
    - callFunction() - Pushes a new stack frame for the function onto the call stack.
    - returnFromFunction() - Pops the top stack frame from the call stack, simulating a function return.
2. **Stack Inspection:**
    - displayCallStack() - Displays the current call stack from top (most recent) to bottom (main), showing the function name and call order.

- getCurrentFunction() - Returns the name of the function at the top of the stack (the currently executing function).

Implementation Guidelines

- Implement a Function and StackFrame class
- You StackFrame class should also have a maximum limit to return false while insertion to simulate a stack overflow error.

Google Test Cases:

```
TEST(CallStackTest, CallSingleFunction) {
    CallStack<int> stack(5); // max size 5
    EXPECT_TRUE(stack.callFunction("main", 1, 0));
    EXPECT_EQ(stack.getCurrentFunction(), "main");
}

TEST(CallStackTest, CallMultipleFunctions) {
    CallStack<int> stack(5);
    stack.callFunction("main", 1, 0);
    stack.callFunction("funcA", 5, 2);
    stack.callFunction("funcB", 10, 1);

    EXPECT_EQ(stack.getCurrentFunction(), "funcB");
}

TEST(CallStackTest, ReturnFromFunction) {
    CallStack<int> stack(5);
    stack.callFunction("main", 1, 0);
    stack.callFunction("funcA", 2, 1);

    EXPECT_EQ(stack.getCurrentFunction(), "funcA");
    EXPECT_TRUE(stack.returnFromFunction());
    EXPECT_EQ(stack.getCurrentFunction(), "main");
}

TEST(CallStackTest, ReturnFromEmptyStack) {
    CallStack<int> stack(5);
    EXPECT_FALSE(stack.returnFromFunction());
}

TEST(CallStackTest, StackOverflow) {
    CallStack<int> stack(2); // max size 2
    EXPECT_TRUE(stack.callFunction("main", 1, 0));
    EXPECT_TRUE(stack.callFunction("funcA", 2, 1));
```

```
    EXPECT_FALSE(stack.callFunction("funcB", 3, 2)); // should fail
}

TEST(CallStackTest, GetCurrentFunctionEmpty) {
    CallStack<int> stack(5);
    EXPECT_EQ(stack.getCurrentFunction(), ""); // No function should be running
}

TEST(CallStackTest, NestedFunctionCallsAndReturns) {
    CallStack<int> stack(5);
    stack.callFunction("main", 1, 0);
    stack.callFunction("funcA", 2, 2);
    stack.callFunction("funcB", 5, 1);

    EXPECT_EQ(stack.getCurrentFunction(), "funcB");
    stack.returnFromFunction(); // returning from funcB
    EXPECT_EQ(stack.getCurrentFunction(), "funcA");

    stack.callFunction("funcC", 7, 3);
    EXPECT_EQ(stack.getCurrentFunction(), "funcC");

    stack.returnFromFunction(); // returning from funcC
    EXPECT_EQ(stack.getCurrentFunction(), "funcA");

    stack.returnFromFunction(); // returning from funcA
    EXPECT_EQ(stack.getCurrentFunction(), "main");
}
```